

第 3 章：MC (Monte Carlo, 蒙特卡洛)、TD (Temporal Difference, 时序差分)、SARSA 与 Q-learning

3.1 本章符号说明

符号/缩写	English	中文	含义
MC	Monte Carlo	蒙特卡洛	用完整 episode 的真实 return 估计价值。
TD	Temporal Difference	时序差分	用 bootstrap target 在线更新价值估计。
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
MDP	Markov Decision Process	马尔可夫决策过程	强化学习中描述状态、动作、转移、奖励和折扣的数学框架。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	更新公式使用的五元组名称。
$S_t / A_t / R_{t+1}$	State / Action / Reward	状态 / 动作 / 奖励	一步交互中的随机变量。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 到 episode 结束的累计收益。
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值估计。
$Q(s,a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值估计。
δ_t	TD Error	时序差分误差	bootstrap target 和当前估计之间的差。
α	Learning Rate	学习率	控制每次更新幅度。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
ϵ	Epsilon	探索概率	epsilon-greedy 中随机探索的概率。
greedy action	Greedy Action	贪心动作	当前估计下 $Q(s,a)$ 最大的动作。
behavior policy	Behavior Policy	行为策略	实际负责采样数据、和环境交互的策略。

符号/缩写	English	中文	含义
target policy	Target Policy	目标策略	算法真正想评估或优化的策略。
on-policy	On-policy Learning	同策略学习	behavior policy 和 target policy 是同一个策略。
off-policy	Off-policy Learning	异策略学习	用 behavior policy 的数据学习另一个 target policy。
RL	Reinforcement Learning	强化学习	本章讨论 model-free RL 的估值和控制方法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	on-policy Monte Carlo policy gradient 算法。
DQN	Deep Q-Network	深度 Q 网络	off-policy value-based deep RL 算法。
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	DDPG 的稳定化改进。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 off-policy actor-critic 算法。
PPO	Proximal Policy Optimization	近端策略优化	on-policy actor-critic 算法。
A2C	Advantage Actor-Critic	优势 Actor-Critic	on-policy actor-critic 算法。

3.2 本章目标

本章进入 model-free RL。你需要掌握：

- Monte Carlo 方法。
- Temporal Difference Learning。
- MC 和 TD 的 bias/variance 区别。
- SARSA。
- Q-learning。
- on-policy 与 off-policy。

3.3 本章主线

第 2 章的 DP 依赖完整环境模型。本章去掉这个假设，只允许 agent 从真实交互中拿到样本。

主线分两步：

1. Prediction：先学会评估一个固定策略的价值。MC 用完整 return 做 target，TD 用一步 reward 加下一状态估计做 bootstrap target。

2. Control：再把价值估计用于改进策略。为了直接比较动作，需要从 $V(s)$ 转到 $Q(s,a)$ 。SARSA 和 Q-learning 不是突然出现的新主题，而是 TD 思想在 action-value control 上的两个版本：SARSA 是 on-policy TD control，Q-learning 是 off-policy TD control。

所以本章的逻辑是：

```
DP exact backup
-> model-free sample backup
-> MC/TD prediction
-> TD control with Q(s,a)
-> SARSA vs Q-learning
```

3.4 Model-free Prediction 的出发点

3.4.1 Model-free 的含义

Model-free 指的是不需要显式知道环境转移模型：

$$P(s' | s, a), R(s, a, s')$$

Agent 只通过和环境交互得到样本：

```
(s, a, r, s')
```

然后用这些样本估计价值函数或优化策略。

3.4.2 统一出发点：价值函数是一个期望

MC 和 TD 都不是凭空来的。它们解决的是同一个问题：价值函数定义里有一个期望，但 model-free 场景下我们没法直接算这个期望。

先看状态价值函数：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

其中 return 是：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

这句话的含义是：

状态 s 的价值，就是在策略 π 下从 s 出发，未来累计折扣 reward 的平均水平。

问题在于这个平均值不能直接看到。我们每次和环境交互，只能看到一条随机轨迹：

```
S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, ...
```

因此从价值定义到算法，核心只有一步：

用样本估计期望。

不同方法的区别在于“用什么样的样本 target”：

方法	可用信息	target 怎么来	代价
DP	知道 $P(s' s,a)$ 和 $R(s,a,s')$	对所有下一状态求精确期望	需要环境模型
MC	不知道模型，但能等 episode 结束	用完整真实 return G_t	等得久，方差大
TD	不知道模型，也不想等结束	用一步 reward 加下一状态估计 $R_{t+1} + \gamma V(S_{t+1})$	有 bootstrap bias

所以 MC 和 TD 可以理解成 DP 的两个 model-free 版本：

- MC：不用模型，也不用下一状态估计，直接等真实 return 出来。
- TD：不用模型，也不等真实 return，用一步采样加当前价值函数估计。

3.5 MC Prediction

3.5.1 Monte Carlo 方法

Monte Carlo 方法用完整 episode 的实际 return 来估计价值。

对于状态价值：

```
V(s) <- average of returns observed after visiting s
```

增量形式：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

其中 G_t 是从时间 t 到 episode 结束的真实累计 return。

特点：

- 不需要环境模型。
- 不需要 bootstrap。
- 必须等 episode 结束才能计算 return。
- 方差通常较大。
- 估计是无偏的，前提是采样足够充分。

First-visit MC：

每个 episode 中，一个状态第一次出现时才用它更新。

Every-visit MC：

每个 episode 中，一个状态每次出现都可以用来更新。

3.5.2 MC 的推理过程

MC 的出发点是价值函数定义：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

这个式子可以拆成一句统计学问题：

我想估计一个随机变量 G_t 在条件 $S_t = s$ 下的均值。

如果我们多次访问同一个状态 s ，就会得到很多个 return 样本：

$$G_t^{(1)}, G_t^{(2)}, \dots, G_t^{(n)}$$

用样本均值估计期望：

$$V_{\pi}(s) \approx 1/n \sum_{i=1}^n G_t^{(i)}$$

这就是 MC prediction 的核心。它不是先写更新公式，而是先把“价值函数”看成“return 的均值”。

如果每次都重新算平均值，写法是：

$$V_n(s) = 1/n \sum_{i=1}^n G_t^{(i)}$$

为了在线更新，可以把样本均值写成递推形式：

$$V_n(s) = V_{n-1}(s) + 1/n [G_t^{(n)} - V_{n-1}(s)]$$

如果不用严格的 $1/n$ ，而用固定学习率 α ，就得到常见写法：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

这里的结构很重要：

```
new estimate = old estimate + step size * (target - old estimate)
```

对 MC 来说：

```
target = G_t
error = G_t - V(s)
```

因此 MC 的完整逻辑是：

1. 从状态 s 出发采样一条 episode。
2. 等 episode 结束后，计算真实 return G_t 。
3. 把 G_t 当作这次对 $V_{\pi}(s)$ 的观测样本。
4. 用样本平均或增量平均更新 $V(s)$ 。

3.5.2.1 为什么 MC target 是无偏的

$$\mathbb{E}_{\pi}[G_t | S_t=s] = V_{\pi}(s)$$

这说明在策略 π 不变、采样充分时， G_t 的期望就是目标价值。MC 用真实 return 做 target，target 本身不依赖当前估计 $V(s)$ ，因此没有 bootstrap bias。

但它的方差可能很大。原因是 G_t 包含整个未来：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

从 $S_t=s$ 开始，后面每一步动作、转移和 reward 都会引入随机性。轨迹越长，return 波动越大。

举个直觉例子：

- 同一个状态 s ，这次 episode 后面走到高奖励区域， G_t 很大。
- 下次 episode 后面因为探索动作走偏， G_t 很小。
- MC 会忠实记录这些完整结果，所以 target 真实，但抖动大。

3.5.2.2 First-visit 和 Every-visit 的区别

如果一个 episode 里同一个状态 s 出现多次：

```
s -> ... -> s -> ... -> terminal
```

First-visit MC 只用第一次访问后的 return 更新。Every-visit MC 每次访问都更新。

面试里一般不需要展开收敛证明，核心记住：

- 两者都在用 return 样本估计期望。
- first-visit 更贴近“每条 episode 给一个独立样本”的直觉。
- every-visit 使用更多样本，但同一 episode 内样本相关性更强。

3.6 TD Prediction

3.6.1 从 MC 过渡到 TD：为什么不想等到 episode 结束

MC 的问题不是思路错，而是工程上有明显限制：

1. 很多任务 episode 很长，等完整 return 才更新太慢。
2. continuing task 没有自然终止， G_t 不容易等出来。
3. 长轨迹 return 方差大，学习曲线抖动。
4. 如果中间某一步已经明显好或坏，MC 也要等结束后才把信号传回前面状态。

TD 的动机就是解决这个问题：

能不能不等完整 G_t ，只走一步就更新？

答案来自 return 的递归分解：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

如果我们知道真实的 G_{t+1} ，那还是要等 episode 结束。但价值函数本身就是对 G_{t+1} 的估计：

$$V_{\pi}(S_{t+1}) = \mathbb{E}_{\pi}[G_{t+1} | S_{t+1}]$$

于是 TD 做了一个替换：

$$G_{t+1} \approx V(S_{t+1})$$

代回去：

$$G_t \approx R_{t+1} + \gamma V(S_{t+1})$$

这就是 TD target 的来源。

3.6.2 TD Learning

Temporal Difference Learning 结合了 Monte Carlo 和 Dynamic Programming 的思想。

TD(0) 更新：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD target：

$$R_{t+1} + \gamma V(S_{t+1})$$

TD error：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

特点：

- 不需要环境模型。
- 不需要等 episode 结束。
- 使用 bootstrap，用当前估计 $V(S_{t+1})$ 更新当前估计。
- 通常方差低于 MC，但可能有 bias。

3.6.3 TD 的推理过程

TD 可以从两个等价角度推出来。

3.6.3.1 角度一：从 Bellman expectation 推出

Bellman expectation equation 是：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

如果有环境模型，DP 可以把这个期望展开成对所有动作和下一状态求和：

$$V_{\pi}(s)=\sum_a \pi(a|s)\sum_{s'} P(s'|s,a)[R(s,a,s')+\gamma V_{\pi}(s')]$$

但 model-free 场景下，我们不知道 $P(s'|s,a)$ ，也不知道完整的转移分布。我们只有一次真实采样：

$(S_t, A_t, R_{t+1}, S_{t+1})$

于是用这一次样本近似 Bellman 期望里的随机变量：

$$R_{t+1}+\gamma V(S_{t+1})$$

这就是 stochastic approximation，即用随机样本近似期望更新。

3.6.3.2 角度二：从 return 递归推出

return 有递归形式：

$$G_t=R_{t+1}+\gamma G_{t+1}$$

MC 使用真实 G_t ，所以必须等未来全部 reward 出来。TD 不等 G_{t+1} ，而是用当前估计替代：

$$G_{t+1}\approx V(S_{t+1})$$

所以 TD target 变成：

$$R_{t+1}+\gamma V(S_{t+1})$$

这一步叫 bootstrap，因为 target 里用了模型自己当前的估计。

3.6.3.3 TD error 的含义

有了 target 后，再用“target 减当前估计”构造误差：

$$\delta_t = R_{t+1}+\gamma V(S_{t+1})-V(S_t)$$

然后按误差方向更新：

$$V(S_t)\leftarrow V(S_t)+\alpha\delta_t$$

如果 $\delta_t>0$ ，说明当前 $V(S_t)$ 低估了，因为一步 reward 加下一状态价值比当前估计更高。

如果 $\delta_t<0$ ，说明当前 $V(S_t)$ 高估了。

3.6.3.4 TD 为什么方差低但有 bias

TD target 只包含一步真实 reward：

$$R_{t+1}$$

后面的长期未来不再用真实随机轨迹，而用估计值：

$$V(S_{t+l})$$

因此 TD target 少受后续随机动作和随机转移影响，方差通常低于 MC。

但 $V(S_{t+l})$ 可能本身不准。target 依赖当前估计，所以会引入 bootstrap bias。

一句话：

MC 的 target 真实但晚、噪声大；TD 的 target 来得早、噪声小，但里面混入了当前价值函数的估计误差。

3.7 Prediction 方法对比与例子

3.7.1 MC vs TD

维度	Monte Carlo	TD
是否需要模型	不需要	不需要
是否需要完整 episode	需要	不需要
是否 bootstrap	否	是
目标	真实 return G_t	$r + \gamma V(s')$
方差	较高	较低
偏差	较低	有 bootstrap bias
适用场景	episodic task	episodic 或 continuing task
更新时机	episode 结束后	每一步之后
信息传播	等完整 return 再回传	reward 可以一步步向前传播

面试表达：

MC 用完整轨迹的真实 return 做 target，不 bootstrap，所以 bias 小但 variance 大。TD 用一步 reward 加下一状态的估计值做 target，能在线更新，variance 小，但因为 target 依赖当前估计，所以会引入 bias。

3.7.2 一个最小数值例子

假设当前估计：

$$V(s)=5, V(s')=8, \gamma=0.9, \alpha=0.1$$

这一步从 s 到 s' ，拿到 reward：

$$R_{t+1}=2$$

TD target 是：

$$2+0.9 \times 8=9.2$$

TD error 是：

$$\delta=9.2-5=4.2$$

更新后：

$$V(s) \leftarrow 5+0.1 \times 4.2=5.42$$

这里没有等 episode 结束。TD 只用了一步真实 reward 和下一状态当前估计。

如果用 MC，则必须等整条 episode 结束。假设最后算出来真实 return 是：

$$G_t=12$$

MC 更新是：

$$V(s) \leftarrow 5+0.1 \times (12-5)=5.7$$

两者差异就在 target：

- MC target 是完整真实 return：12。
- TD target 是一步 bootstrap：9.2。

这个例子说明：TD 不一定比 MC 更准，它只是更早、更低方差地更新。

3.7.3 本章例子：什么时候用 MC，什么时候用 TD

3.7.3.1 例子 1：Blackjack 适合解释 MC

Blackjack 是一个 episodic task：

- 一局牌开始。
- 玩家要牌或停牌。
- 最后赢、输或平局。
- episode 很短，结束明确。

这时 MC 很自然。比如玩家在某个状态 s 做决策后，等这一局结束就能知道真实结果：

```
赢了: G_t = +1
输了: G_t = -1
平局: G_t = 0
```

多玩很多局后，访问同一个状态得到很多 return 样本，用平均值估计这个状态价值。

面试讲法：

Blackjack 适合 MC，因为 episode 短、终止清晰、完整 return 容易拿到。MC 不需要知道牌堆转移概率，只要反复采样完整对局。

3.7.3.2 例子 2：在线广告更适合解释 TD

假设系统每次给用户推荐一个广告，用户状态会持续变化，没有明确“游戏结束”。

如果用 MC，可能要等用户一整天甚至一周的长期收益才更新，这太慢。

TD 可以每一步更新：

当前状态：用户刚打开 App
 动作：展示广告 A
 即时 reward：点击 +1，没有点击 0
 下一状态：用户继续浏览或离开
 TD target = 即时 reward + 下一状态价值估计

这说明 TD 的优势：

- 不需要等完整长期结果。
- 每次交互后都能更新。
- 适合 continuing task 或长 episode 任务。

3.8 从 Prediction 到 TD Control

3.8.1 从 Prediction 到 Control：为什么接 SARSA 和 Q-learning

到这里为止，MC 和 TD 主要解决的是 prediction：

给定一个策略 π ，估计它的价值。

但强化学习最终要做 control：

找到一个更好的策略。

两者的区别是：

任务	学什么	策略是否固定	典型问题
V prediction	$V_{\pi}(s)$	固定	当前策略下，状态 s 值多少钱
Q prediction	$Q_{\pi}(s,a)$	固定	当前策略下，在 s 做 a 值多少钱
control	$Q(s,a)$ 并改进 π	不固定	在 s 应该选哪个动作

为什么 control 常从 $Q(s,a)$ 入手？

如果已知环境模型，可以用 $V(s)$ 加上 $P(s'/s,a)$ 和 $R(s,a,s')$ 做一步 lookahead：

$$\sum_s P(s'/s,a)[R(s,a,s')+\gamma V(s')]$$

但 model-free 场景下，模型未知，agent 不能直接对所有下一状态求和。此时更自然的是直接学习每个动作的长期价值：

$$Q(s,a)$$

有了 $Q(s,a)$ ，策略改进就很直接：

choose action with larger $Q(s, a)$

所以 SARSA 和 Q-learning 的位置应该理解为：

TD prediction learns $V_{\pi}(s)$
 TD control learns $Q(s,a)$ and improves policy
 SARSA and Q-learning are two TD control algorithms

它们都沿用 TD 的更新骨架：

new estimate = old estimate + alpha * (TD target - old estimate)

区别只在 TD target 怎么定义：

- SARSA：target 跟随实际行为策略采样到的下一个动作。
- Q-learning：target 假设下一步执行 greedy action。

3.8.2 先定义：greedy、epsilon-greedy、on-policy、off-policy

下面马上要讲 SARSA 和 Q-learning。这里先把几个词定义清楚，否则后面的“因此是 on-policy / off-policy”没有意义。

3.8.2.1 Greedy action / greedy policy

Greedy action 指当前 $Q(s,a)$ 估计下价值最大的动作：

$$a_{\text{greedy}} = \operatorname{argmax}_a Q(s,a)$$

Greedy policy 指每个状态都选择当前看起来最好的动作：

choose $\operatorname{argmax}_a Q(s, a)$

它的问题是：如果早期 Q 估计错了，agent 可能一直选错动作，永远不探索其他动作。

3.8.2.2 Epsilon-greedy behavior policy

Epsilon-greedy 是一种带探索的行为策略。 ϵ 表示随机探索的概率：

```

with probability epsilon:
    choose a random action
otherwise:
    choose argmax_a Q(s, a)

```

例如 $\epsilon=0.1$ ：

- 10% 概率随机选动作，用来探索。
- 90% 概率选当前 $Q(s,a)$ 最大的动作，用来利用当前知识。

所以 epsilon-greedy 不是一个新算法，而是一种“采样数据时怎么选动作”的策略。

3.8.2.3 Behavior policy 和 target policy

在 control 问题里要分清两个策略：

名词	中文	含义
behavior policy	行为策略	实际和环境交互、产生样本的策略。
target policy	目标策略	算法真正想评估或优化的策略。

如果 agent 用 epsilon-greedy 和环境交互，那么 behavior policy 就是 epsilon-greedy。

如果算法更新时假设“未来都选 Q 最大的动作”，那么 target policy 就是 greedy policy。

3.8.2.4 On-policy 和 off-policy

On-policy，中文可以叫同策略学习：

用当前实际采样的 behavior policy，同时作为学习目标。

也就是：

$$\pi_{behavior} = \pi_{target}$$

如果 behavior policy 是 epsilon-greedy，on-policy 算法学到的就是“我继续按这套 epsilon-greedy 策略行动时，长期价值是多少”。

Off-policy，中文可以叫异策略学习：

用一个 behavior policy 采样数据，但学习另一个 target policy。

也就是：

$$\pi_{behavior} \neq \pi_{target}$$

例如：用 epsilon-greedy 采样，保持探索；但更新 target 时假设下一步走 greedy action。这个就是 off-policy。

一句话判断：

判断问题	结论
target 用实际采样到的下一个动作 A_{t+1} 吗？	通常是 on-policy。
target 跳过实际动作，直接用 $\max_a Q(s',a)$ 吗？	通常是 off-policy。

3.9 TD Control 算法

3.9.1 TD Control 1: SARSA

现在再看 SARSA。SARSA 是 on-policy TD control 方法。

这里的 on-policy 含义是：SARSA 用当前行为策略采样数据，也学习这个当前行为策略本身的价值。如果行为策略是 epsilon-greedy，那么 SARSA 学的就是“继续按这套 epsilon-greedy 策略行动时，长期回报是多少”。

SARSA 名字来自一次更新用到的五元组：

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$$

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

关键点：

- A_{t+1} 是 behavior policy 在下一状态真实采样出来的动作。
- 如果 behavior policy 是 epsilon-greedy，那么 A_{t+1} 可能是 greedy action，也可能是随机探索动作。
- SARSA 的 target 使用这个真实采样的 A_{t+1} ，所以它学习的 target policy 就是当前 behavior policy。
- 因此 SARSA 是 on-policy：采样用谁，学习目标就是谁。

直觉：

SARSA 学的是“我按照当前这套带探索的策略行动，长期回报会是多少”。

3.9.2 SARSA 的 target 为什么是 $Q(s',a')$

对 action-value 来说，Bellman expectation 可以理解成：

$$Q_{\pi}(s,a) = \mathbb{E}[R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1})]$$

SARSA 在采样时真的执行了下一步动作 A_{t+1} 。如果当前 behavior policy 是 epsilon-greedy，那么 A_{t+1} 可能是 greedy action，也可能是随机探索动作。

所以 SARSA 的 target 是：

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

它把“探索可能造成的后果”也学进去。这就是为什么在 Cliff Walking 里 SARSA 更保守：它知道自己未来还可能因为 ϵ 探索而掉下去。

3.9.3 TD Control 2: Q-learning

Q-learning 是 off-policy TD control 方法。

这里的 off-policy 含义是：Q-learning 可以用 epsilon-greedy 这种 behavior policy 去探索和采样，但它更新时学习的是 greedy target policy，也就是“未来都选当前 Q 最大动作”的策略。

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

关键点：

- 行为时可以用 epsilon-greedy 探索。
- 更新 target 时直接用 $\max_a Q(s', a)$ 。
- 学习目标是 greedy policy 的价值，而不完全等于行为策略。
- 因此 Q-learning 是 off-policy：采样策略可以带探索，但学习目标是 greedy policy。

直觉：

Q-learning 学的是“如果未来都选当前估计下最优的动作，长期回报会是多少”。

3.9.4 Q-learning 的 target 为什么用 max

Q-learning 对应的是 Bellman optimality 思路：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a Q^*(S_{t+1}, a')]$$

model-free 场景下，把期望用一次 transition 近似，就得到 target：

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a')$$

这里要分清两件事：

- 行为时：可以用 epsilon-greedy 收集数据，保留探索。
- 学习时：target 假设未来执行 greedy action。

所以 Q-learning 是 off-policy。它不是在评估“实际带探索的行为策略”，而是在用探索数据学习一个 greedy target policy 的价值。

面试里可以这样推：

SARSA 从 Bellman expectation 来，下一动作就是行为策略实际采样的 a' ；Q-learning 从 Bellman optimality 来，下一动作取当前 Q 最大的动作。因此 SARSA on-policy，Q-learning off-policy。

3.9.5 SARSA vs Q-learning

维度	SARSA	Q-learning
策略类型	on-policy	off-policy
target	$r + \gamma Q(s', a')$	$r + \gamma \max_a Q(s', a)$
是否考虑探索动作影响	是	否
风格	更保守	更贪心
学习目标	当前行为策略	最优 greedy 策略

3.9.6 本章例子：Cliff Walking 解释 SARSA 和 Q-learning

Cliff Walking 中，起点到终点的最短路径贴着悬崖走，但 epsilon-greedy 探索可能让 agent 随机走错一步掉下去。

SARSA 学当前带探索行为策略的价值，所以它会把“未来仍可能探索并掉下悬崖”的风险计入 target：

```
r + gamma * Q(s', actual a')
```

因此 SARSA 往往学到更保守、更远离悬崖的安全路径。

Q-learning 学 greedy target policy 的价值。虽然行为时仍用 epsilon-greedy 探索，但更新 target 假设下一步会选择当前最优动作：

```
r + gamma * max_a Q(s', a)
```

因此 Q-learning 更容易学到贴近悬崖的最短路径。

这个例子对应的逻辑是：

- SARSA 会考虑 epsilon-greedy 探索可能掉下悬崖，因此倾向选择更安全路径。
- Q-learning 更新时假设未来总能选最优动作，因此更倾向学习贴近悬崖的最短路径。

面试讲法：

SARSA 和 Q-learning 都是 TD control。SARSA 的 target 来自行为策略实际采样到的下一动作，所以是 on-policy；Q-learning 的 target 来自 greedy max，所以是 off-policy。Cliff Walking 中 SARSA 更保守，Q-learning 更贪心。

3.9.7 On-policy vs Off-policy 总结

前面已经先定义过一次，这里只做总结。

On-policy：

学习和评估的是当前实际采样的行为策略。

$$\pi_{behavior} = \pi_{target}$$

例如：

- SARSA
- REINFORCE
- A2C
- PPO

Off-policy：

用一种行为策略收集数据，但学习另一种目标策略。

$$\pi_{behavior} \neq \pi_{target}$$

例如：

- Q-learning
- DQN
- DDPG
- TD3
- SAC

off-policy 的优势：

- 可以复用历史数据。
- sample efficiency 通常更高。
- 可以从 replay buffer 学习。

off-policy 的难点：

- 分布不匹配。
- 重要性采样或 value correction 可能带来高方差。
- 结合函数逼近和 bootstrap 时可能不稳定。

3.10 本章面试高频问题

3.10.1 MC 和 TD 的区别？

MC 用完整 episode 的真实 return 更新，不 bootstrap，bias 小但 variance 大。TD 用 $r + \gamma V(s')$ 这种一步 bootstrap target 更新，可以在线学习，variance 小但有 bias。

3.10.2 SARSA 和 Q-learning 的区别？

SARSA 是 on-policy，target 使用实际采样到的下一个动作 a' 。Q-learning 是 off-policy，target 使用 $\max_a Q(s', a)$ ，学习 greedy target policy。

3.10.3 SARSA/Q-learning 和 TD 有什么关系？

SARSA 和 Q-learning 都是 TD control。TD prediction 更新 $V(s)$ ，用于评估固定策略；SARSA/Q-learning 更新 $Q(s, a)$ ，用于边估值边改进策略。两者都使用一步 bootstrap target，只是 SARSA 用实际采样的 a' ，Q-learning 用 greedy max。

3.10.4 为什么 Q-learning 是 off-policy?

因为采样数据可以来自 epsilon-greedy 行为策略，但更新目标是假设下一步选择 greedy action 的价值，即 target policy 和 behavior policy 不同。

3.10.5 TD error 是什么?

TD error 是当前价值估计和 bootstrap target 之间的差：

$$\delta = r + \gamma V(s') - V(s)$$

它衡量当前估计需要被修正的方向和大小。

3.10.6 为什么 TD 可以不等 episode 结束?

因为 TD target 只需要一步 reward 和下一状态的当前价值估计，不需要完整轨迹 return。

3.11 本章练习

1. 手写 SARSA 和 Q-learning 的更新公式。
2. 解释为什么 SARSA 在 cliff walking 中更保守。
3. 写一个 tabular Q-learning 的伪代码。
4. 用 bias/variance 角度比较 MC 和 TD。

[章节索引](#)

[← 上一章](#) [下一章 →](#)